SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA-1709

DONE BY: VISHAL BALAJI J (192525014)

CONSTRAINT-BASED PROBLEM SOLVING

QUESTION 1: DOCTOR SHIFT ASSIGNMENT USING BACKTRACKING SEARCH

The task is to assign doctors D1, D2 and D3 to the Morning, Afternoon and Night shifts. Each doctor must receive exactly one shift, and each shift must contain exactly one doctor. D1 cannot work the Night shift, D2 must work before D3, and D3 cannot work the Morning shift. Backtracking Search assigns a shift to one doctor at a time, checks the constraints immediately, and returns to an earlier decision if a constraint is violated.

Step 1: Assign D1 to Morning. This is valid because D1 is not assigned to Night. Step 2: Assign D2 to Afternoon. This shift is still available, and D2 can work before D3. Step 3: Assign D3 to Night. D3 is not assigned to Morning, and D2's Afternoon shift occurs before D3's Night shift. All shifts are distinct and every doctor has one shift, so the assignment satisfies every constraint and no backtracking is required.

Step	Decision	Constraint check
1	D1 → Morning	Valid: D1 is not on Night.
2	D2 → Afternoon	Valid: shift is unused; D2 is before D3.
3	D3 → Night	Valid: D3 is not on Morning and follows D2.
Result	Morning: D1; Afternoon: D2; Night: D3	All constraints are satisfied.

Final answer: The valid schedule is Morning—D1, Afternoon—D2, and Night—D3. Each doctor has exactly one shift, no shift is shared, D1 avoids Night, D3 avoids Morning, and D2 works before D3.

QUESTION 2: 5 × 5 GRID NAVIGATION USING BFS

Breadth-First Search (BFS) explores all reachable cells at distance 0, then distance 1, distance 2, and so on. It uses a first-in, first-out (FIFO) queue. Because every movement costs 1, the first time BFS reaches the goal it has found a path with the minimum number of moves. The robot can move Up, Down, Left or Right, cannot enter blocked cells, and records each cell's parent so the route can be

reconstructed. Manhattan distance is h(n) = |row(n) − row(G)| + |column(n) − column(G)|. It estimates the remaining distance; standard BFS does not use this heuristic to order its queue.

Grid used for the worked solution (S = start, G = goal, # = obstacle, . = free cell):

S	.	#	.	.
#	.	#	.	#
.	.	.	.	.
.	#	#	#	.
.	.	.	.	G

Coordinates use (row, column), starting at (0,0). S = (0,0), G = (4,4). Neighbours are examined in this fixed order: Right, Down, Left, Up. A cell is marked visited when it is first enqueued, preventing duplicate queue entries.

Step-by-step node expansion and queue updates

Expansion 1: remove (0, 0) (cost 0). Newly enqueued: (0, 1) (cost 1), (1, 0) (cost 1). Queue after expansion: (0, 1), (1, 0).

Expansion 2: remove (0, 1) (cost 1). Newly enqueued: (1, 1) (cost 2). Queue after expansion: (1, 0), (1, 1).

Expansion 3: remove (1, 0) (cost 1). Newly enqueued: No new nodes. Queue after expansion: (1, 1).

Expansion 4: remove (1, 1) (cost 2). Newly enqueued: (2, 1) (cost 3). Queue after expansion: (2, 1).

Expansion 5: remove (2, 1) (cost 3). Newly enqueued: (2, 2) (cost 4). Queue after expansion: (2, 2).

Expansion 6: remove (2, 2) (cost 4). Newly enqueued: (2, 3) (cost 5). Queue after expansion: (2, 3).

Expansion 7: remove (2, 3) (cost 5). Newly enqueued: (2, 4) (cost 6), (1, 3) (cost 6). Queue after expansion: (2, 4), (1, 3).

Expansion 8: remove (2, 4) (cost 6). Newly enqueued: (3, 4) (cost 7). Queue after expansion: (1, 3), (3, 4).

Expansion 9: remove (1, 3) (cost 6). Newly enqueued: (0, 3) (cost 7). Queue after expansion: (3, 4), (0, 3).

Expansion 10: remove (3, 4) (cost 7). Newly enqueued: (4, 4) (cost 8). Queue after expansion: (0, 3), (4, 4).

Expansion 11: remove (0, 3) (cost 7). Newly enqueued: (0, 4) (cost 8). Queue after expansion: (4, 4), (0, 4).

Expansion 12: remove (4, 4) (cost 8). Newly enqueued: No new nodes. Queue after expansion: (0, 4).

Optimal path: (0, 0) → (0, 1) → (1, 1) → (2, 1) → (2, 2) → (2, 3) → (2, 4) → (3, 4) → (4, 4).

Total path cost: 8 moves × 1 per move = 8. The path is optimal because BFS processes all nodes in increasing order of depth and reaches the goal at the smallest possible number of moves.